

Low-delay cost-aware multipath scheduling over dynamic links for access traffic steering, switching, and splitting

Marcus Pieska^{a,*}, Alexander Rabitsch^a, Anna Brunstrom^a, Andreas Kessler^{a,c}, Markus Amend^b, Eckard Bogenfeld^b

^a Karlstad University, Computer Science Department, 65188 Karlstad, Sweden

^b Deutsche Telekom, 64295 Darmstadt, Germany

^c Deggendorf Institute of Technology, Department of Applied Computer Science, 94469 Deggendorf, Germany

ARTICLE INFO

Keywords:

Heterogeneous wireless access
5G
ATSSS
Multi-path transport layer tunneling
Unreliable traffic
MP-DCCP
Transport layer
Packet scheduling

ABSTRACT

Bundling of multiple access technologies is currently being standardized by 3GPP in the 5G *access traffic steering, switching and splitting* (ATSSS) framework, with the goal to increase robustness, resiliency and capacity of wireless access. A key part of an ATSSS framework is the packet scheduler, which decides the access network over which each packet is to be transmitted. As wireless channels are highly dynamic, a challenge for any scheduler is to correctly estimate the capacity of each path, and thereby avoid congesting the paths. In this paper, we further develop a recent packet scheduler that exploits cross-layer information from the congestion control state of individual transport layer tunnels when making scheduling decisions. Our aim is to achieve good path utilization while keeping the congestion delay low. Extensive emulations show that our approach reduces the excess delay at the bottleneck to as little as 34%. We furthermore show that our approach improves the performance of end-to-end applications including WebRTC and YouTube compared to state-of-the-art.

1. Introduction

Current forecasts [1] predict that the amount of mobile data traffic will increase drastically, from roughly 84 exabytes per month by the end of 2021 to 368 exabytes per month by 2027. *Fifth Generation* (5G) cellular networks are currently being rolled out to help close the gap between current cellular capacity and expected near future mobile traffic demand. In order to increase the available capacity, resiliency and robustness of wireless access networks, 5G system architectures include *built-in multipath support* over multiple wireless technologies, including *non-trusted* wireless access like WiFi.

Transport-level multi-connectivity in 5G may be implemented in *Above-the-Core* or *Core-Centric* manners [2]. *Above-the-Core* entails an end-to-end multipath connection between the *User Equipment* (UE) and any server endpoint. While this does not require changes to existing network equipment, it requires that *all* endpoints (clients and servers) implement a multipath transport protocol. The *Core-Centric* approach entails a proxied multipath connection between the UE and the 5G core. Such solutions are more complex since they need additional in-network support functions. However, they allow for operator control over traffic distribution, support any existing client and server applications, and are thus *transparent* to content providers. As legacy

systems and applications may be difficult to change, transparency is particularly desirable, as it potentially allows *any* application flow to enjoy multi-connectivity, even if the end-to-end protocol does not support it.

The *Access Traffic Steering, Switching, and Splitting* (ATSSS) architecture – see 3GPP Rel. 16 [3] – aims to standardize different solution approaches for transport-layer multi-connectivity in 5G. In this context, the *steering* function refers to the capability to select the best path to use for a particular flow according to its service requirements, the *switching* function refers to the capability of performing seamless handovers between different paths without causing an interruption in the service, and the *splitting* function refers to the aggregation of path capacities. There are several proposals on how ATSSS can be implemented [2]. One approach is to use a proxy inside the core network, e.g. at the User Plane Function (UPF), which would enable the UE to communicate transparently over multiple access networks, even if the remote end-host does not support a multipath-capable transport protocol. By using an ATSSS proxy, (i) TCP flows may be split at the proxy and turned into *Multipath TCP* (MPTCP) [4] between the proxy and the user. Alternatively, at the same proxy (ii) non-TCP traffic may instead be

* Corresponding author.

E-mail addresses: marcus.pieska@kau.se (M. Pieska), alexander.rabitsch@kau.se (A. Rabitsch), anna.brunstrom@kau.se (A. Brunstrom), andreas.kessler@kau.se, andreas.kessler@th-deg.de (A. Kessler), markus.amend@telekom.de (M. Amend), eckard.bogenfeld@telekom.de (E. Bogenfeld).

<https://doi.org/10.1016/j.comnet.2024.110186>

Received 17 February 2023; Received in revised form 30 August 2023; Accepted 6 January 2024

Available online 11 January 2024

1389-1286/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

encapsulated or *tunneled over a multipath-capable protocol* [5], e.g. using *Multipath DCCP* (MP-DCCP) [6], *Multipath QUIC* (MP-QUIC) [7] [8] together with the datagram extension [9], or concurrent multipath transfer SCTP (CMT-SCTP) [10].

When splitting a flow over multiple paths, the decision of which path to use is crucial for the achievable performance and reliability of the network. This decision is made by a *packet scheduler* [11–14], which determines for each packet the path over which it should be transmitted, based on the policy of the scheduler. The *strict priority scheduler* [11] transmits over a preferred path (e.g. WiFi) until its capacity is fully used, at which point it will send packets over the lower priority secondary path (e.g. 4G). Paths may be prioritized based on various criteria, e.g. *economic aspects*. As ATSSS aims to offer cellular users aggregation of several access technologies, including WiFi, a scheduler that prioritizes the cheapest alternative becomes attractive. Both cellular users and operators alike could benefit from off-loading traffic onto WiFi. The *Cheapest Path First* (CPF) scheduler is an implementation of the strict priority scheduler, designed to deliver this kind of offloading.

Our previous work [14] has shown that although CPF prioritizes the cheaper path when used in a multipath-tunnel context, the scheduler also often fails to utilize the expensive path *despite* there being demand for the aggregated capacity. This is a consequence of nested *Congestion Control* (CC), where the *tunnel CC* – *between the UE and proxy* – congests the primary path such that the *end-to-end CC* reacts and lowers the sending rate before packets can overflow onto the secondary path. Consequently, the *Adaptive CPF* (ACPF) scheduler was proposed in our previous paper [14] to solve this issue, by utilizing the secondary path at an earlier point, before congestion on the primary path becomes too excessive.

Wireless channels, e.g. millimeter wave channels [15], can vary dramatically in capacity over short time intervals. Similar, but less drastic capacity variation may occur in traditional WiFi deployments when users roam around and the connection to the access point suddenly breaks, leading to longer periods of disconnection. Examples of such capacity variation for WiFi can be seen from our measurements in Fig. 5. Sudden capacity variation calls for a *tighter integration* between the scheduler and the CC algorithm (CCA), or a *shorter CC loop* [15,16] to enable timely reactions to sudden capacity changes. However, ACPF has previously only been evaluated over static links, and only for a limited set of traffic types. Consequently, it is important to understand the performance of ACPF under dynamic conditions, when the available bandwidth of access networks changes more rapidly. In addition, the impact of ACPF on delay-sensitive applications must be understood further. This is of particular importance as non-TCP traffic may be latency sensitive, and will include real-time application traffic. Additionally, the multipath proxy is typically *closer to the UE than the server* is. Therefore, it will likely schedule traffic more effectively than end-to-end multipath scheduling as a scheduler at the proxy can react faster to capacity changes. Therefore, ATSSS is expected to benefit from having proxied multipath tunnels for non-TCP traffic for which a split approach is not feasible, given there are packet scheduling solutions that are able to effectively aggregate the capacity of the different links *while maintaining low latency*.

Our contributions in this paper are the following: (i) we show that CPF cannot properly utilize links with dynamic capacities, as interactions between the packet scheduler and transport layer tunnel congestion controls will lead to *high latency*; (ii) we improve the performance of ACPF by utilizing cross-layer information from transport layer BBR tunnels, and show that our improved scheduler can handle dynamic links better when it is allowed to exploit the BBR path model; (iii) we use trace-based network emulations to evaluate the achievable performance of latency sensitive applications and media streaming protocols over an ATSSS tunneling framework based on MP-DCCP, using both user and kernel space implementations of our ACPF extensions; (iv) we demonstrate that our ACPF scheduler effectively aggregates

path capacities while keeping the per path latencies low; (v) we also show that better scheduling is achieved if the proxy is close to the user (e.g. edge deployment), rather than further away (e.g. cloud deployment). Our evaluation shows that our approach reduces excess delay at the bottleneck to as little as 34%, while end-to-end performance improves significantly for SRT, WebRTC, and YouTube when using the improved version of ACPF.

The rest of the paper is structured as follows. Section 2 introduces background and related work. Section 3 describes CPF, illustrates performance problems and introduces the improved ACPF scheduler. Section 4 provides details on the new ACPF implementation in a user-space multi-path framework, as well as its integration into kernel-space MP-DCCP. Section 5 describes our evaluation setup. Section 6 presents the results of our evaluation; and finally, Section 7 concludes the paper.

2. Background and related work

In this section, we introduce the necessary background information and related works.

2.1. Access Traffic Steering, Switching, and Splitting

ATSSS is specified in 3GPP Rel. 16 [3]. It enables the simultaneous use of 3GPP access networks and non-3GPP access networks, where the latter may or may not be controlled by the mobile network operator — e.g. WiFi. ATSSS provides a number of benefits to users and mobile network operators alike. Users may enjoy an improved user experience due to the capabilities of ATSSS to utilize multiple access networks in parallel, thus aggregating the available wireless capacity. Mobile network operators can expect an improved flexibility in how their resources are utilized. The convergence of access networks as provided by ATSSS not only provides operators an opportunity to reduce the pressure on their cellular networks by offloading traffic to the non-3GPP access, but also enables a reduction in the architectural complexity of their networks, especially in the case when the network operator has full control of both the non-3GPP and 3GPP accesses.

Fig. 1 illustrates the high-level architecture for ATSSS. The *Policy Control Function* (PCF) defines *Policy and Charging Configuration* (PCC) rules, which are translated by the *Session Management Function* (SMF) into ATSSS rules shared with the *User Plane Function* (UPF), and the UE via the *Access and Mobility Management Function* (AMF). These rules define how traffic should be distributed across access networks. Such traffic distribution is at the core of ATSSS and needs to be implemented by both the UPF (for downlink traffic) and the UE (for uplink traffic) providing a converged network architecture for fixed and mobile core networks. An ATSSS rule includes a traffic descriptor that indicates for which type of traffic the rule applies, and one of the following *steering modes* [17]: (i) *active-standby*, which uses the *active* path if available, and otherwise resorts to the *standby* path; (ii) *smallest-delay*, where traffic is steered to the path with the shortest RTT; (iii) *load balancing*, where a predefined percentage of the traffic is steered across both paths simultaneously; or (iv) *priority based* – i.e. *strict priority* [11] – where the traffic is steered over the access with the highest priority until its capacity is fully utilized, after which the secondary path may be used. Our interest is in the last of these steering modes, and in particular when it is used to e.g. utilize the red WiFi path in Fig. 1 first, while utilizing the blue cellular path only when the WiFi is fully utilized.

In order to implement such steering modes, the ATSSS framework requires some mechanism for distributing packets over the available paths, i.e. a packet scheduler. An early evaluation of packet schedulers [18] revealed that (i) *head-of-line blocking* and (ii) *receive window blocking* may negatively impact the performance of MPTCP. The SRTT scheduler [19] is the default MPTCP scheduler, which prioritizes the path with the shortest SRTT until there is no more space in its *congestion window* (CWND), after which the scheduler moves

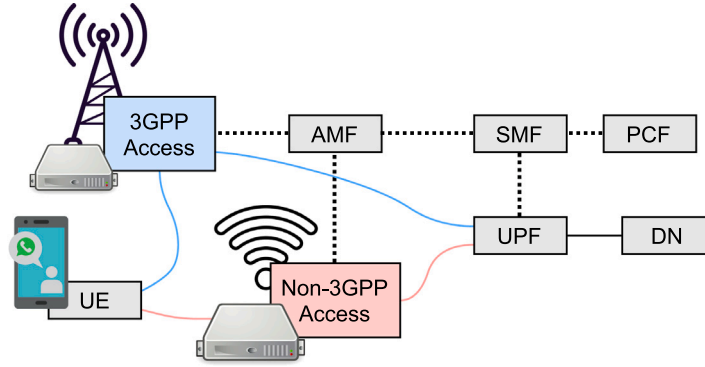


Fig. 1. High-level architecture for the ATSSS service.

on to the path with the second shortest SRTT, and so on. This behavior usually leads to the aforementioned blocking issues, as the packets tend to be received out-of-order on the receiver side. This effect is accentuated by highly asymmetric paths. A number of schedulers have therefore been designed to tackle head-of-line and receive window blocking, including *Out-of-Order Transmission for In-Order Arrival Scheduling* (OTIAS) [20], *Blocking Estimation-based MPTCP scheduling* (BLEST) [21], *Earliest Completion First* (ECF) [22], *Decoupled Multipath Scheduler* (DEMS) [23], and *Peekaboo* [12].

Schedulers require information on the current characteristics of each path, such as latency and available bandwidth, in order to operate effectively. An ATSSS service enabled by MPTCP can get such information directly from the *Congestion Control* (CC) state of its subflows. Non-TCP traffic cannot easily be transformed into a multipath-capable protocol the same way TCP can be turned into MPTCP. However, a *multipath tunnel* is one alternative to the aforementioned connection-splitting approach. Such tunnels can achieve ATSSS functionality independent of the end-to-end transport layer protocol, while also providing the means to gather information on the current network state of each path. However, this approach also presents challenges, as the tunnel introduces overhead, and the interactions between *inner* and *outer* congestion control loops are often complex.

2.2. Congestion control algorithms

Congestion Control Algorithms (CCAs) have two main goals: maximizing the sending rate and avoiding network congestion. Most CCAs tend to regulate the number of in-flight packets to be no more than the *congestion window* (CWND). TCP-NewReno [24] is an early CCA that employs an *Additive Increase/Multiplicative Decrease* (AIMD) scheme. For each RTT, TCP-NewReno increases the CWND with one packet, until a loss is detected, at which point the CWND is reduced by half. TCP-Cubic [25], as its name implies, increases its CWND according to a cubic function. The main idea is to find the maximum capacity of the path faster than TCP-NewReno, while plateauing at a CWND that does not induce congestion. In contrast to loss-based algorithms, TCP-BBR [26] is designed to actively probe for the maximum bottleneck bandwidth and the minimum RTT, from which the BDP is estimated. TCP-BBR then sets CWND equal to roughly two times the BDP. This factor is often referred to as the *gain* of BBR, and while it is typically set to approximately 2, some implementations may use other values. CCID5 [27] is an implementation of the TCP-BBR algorithm for the DCCP protocol; DCCP may be likened to a congestion-controlled variant of UDP. Although there are some subtle differences between TCP-BBR and CCID5, we will refer to both CCAs simply as *BBR* henceforth. Estimating the BDP when using a CCA other than BBR requires the continuous tracking of the minimum RTT, RTT_{min} , and limiting CWND such that the RTT does not exceed RTT_{min} [28].

In order to tackle fairness concerns, *coupled* CCAs [29] may be used for multipath protocols. Such CCAs couple the CWND growth of the

connection subflows such that the combined traffic distributed over the paths does not take up more than its fair share over potentially shared bottleneck links. In ATSSS, it is reasonable to assume that the different paths are disjunct, only converging at the multipath proxy, since the ATSSS architecture aims to only aggregate the capacity of the *access networks*. Moreover, depending on the wireless standard, wireless edge nodes tend to have per-user queues and some measure of fair-queuing. Therefore, coupled congestion control is not a necessity for ATSSS.

3. Motivation and design

In this section we introduce a previously observed problem [14] with CPF in Section 3.1; we present the high level design of ACPFv1 [14] in Section 3.2; we present and motivate our proposed improvements for ACPFv2 in Section 3.3; we compare the two versions of ACPF in Section 3.4; finally, we give the rationale for some non-obvious design decisions and address the competitiveness of ACPFv2 in Section 3.5.

3.1. CPF performance problems

The goal of a strict priority scheduler – e.g. *CPF* – is to first fully utilize the primary – e.g. *cheap* – path fully before scheduling packets on the secondary – e.g. *expensive* – path. CPF uses information from the congestion state of a path to decide if the path is fully used; i.e., if the number of in-flight packets matches or exceeds the size of the path CWND, then the path is perceived as fully used. In the multipath protocols we use, each path has its own uncoupled congestion control and CWND.

When CPF is used in combination with congestion-controlled multipath tunnels, e.g. within the ATSSS framework for non-TCP traffic, it is difficult for the scheduler to effectively aggregate the capacity of both paths. The second path will in some cases hardly be used at all. The reason for the poor aggregation is due to interactions between inner and outer congestion control loops in the multi-path tunnel. The left side of Fig. 2 shows a simplified example of how a server CWND grows over time, and where that CWND is distributed over a multipath system using CPF. If the tunnel CCA congests the cheap path, the end-to-end server CWND will tend to end up in the *bottleneck buffer* on the cheaper path, as opposed to being *in-flight* over the expensive path. Due to queuing at the bottleneck buffer on the cheap path, delay increases. The delay increase that CPF will cause before utilizing the expensive path is our *primary interest*. Additionally, if the end-to-end flow uses a delay-based CCA, the congestion of the cheaper path may cause the end-to-end flow to decrease the size of its CWND prematurely, while there is still unutilized capacity on the expensive path.

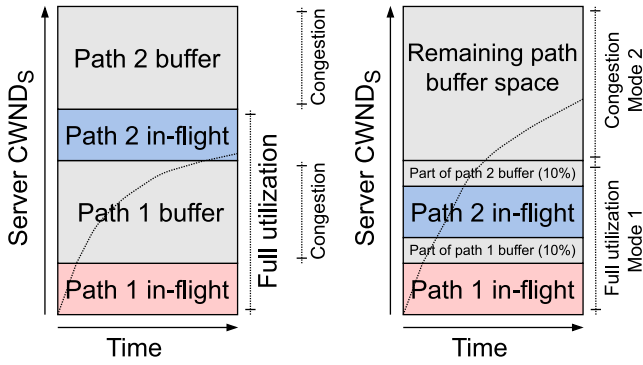


Fig. 2. Server $CWND$, growth & distribution over a multipath framework; path 1 is cheaper, while path 2 is more expensive; CPF (left) tends to cause congestion – distribute $CWND$ in gray areas – before reaching full utilization of all paths – distribute $CWND$ in red and blue areas; ACPFv2 (right) sets a delay target – 10% relative to the minimum path RTT – that has to be reached for both paths before the full $CWND$ may be used; ACPFv1 is not depicted but operates in a manner similar to ACPFv2 but with an implicit target delay, in principle at 0% but in practice being arbitrary but still small.

3.2. The ACPFv1 scheduler

Adaptive Cheapest Path First (ACPFv1) is a redesign of CPF with the idea to use the expensive path earlier compared to CPF, at a point when the cheaper path is less congested. ACPFv1 [14] has previously been shown to enable *better capacity aggregation than CPF*, but the impact on delay has not been evaluated. ACPFv1 tries to make a more intelligent decision as to when the cheap path is considered *fully utilized*, and consequently when ACPFv1 should start utilizing the expensive path. ACPFv1 considers a path fully utilized once it has reached a number of in-flight packets presumed to correspond to ~ 1 non-congested BDP. The term non-congested BDP refers to the BDP attained when multiplying the bottleneck throughput with the lowest possible RTT , any more packets scheduled over a path in such a state will likely contribute to *congestion at the path bottleneck*. As the end-to-end $CWND$ grows, ACPFv1: (i) first increases the number of packets in-flight over the cheaper path until it reaches the BDP of that path; (ii) then starts using the expensive path until the number of packets in-flight reaches the BDP of that path; (iii) then allows usage of the full $CWND$ of the cheaper path. Note that default CPF is liable to reverse the order of points two and three. This process is *similar* to the right side of Fig. 2; however, the right side *actually describes our improved ACPFv2 scheduler*.

BBR is a congestion control algorithm that is available in TCP, as well as in DCCP under the name CCID5 [27]. BBR maintains a model of the path by tracking the throughput and the minimum RTT , from which BBR may calculate the non-congested BDP. ACPFv1 [14] leverages the path model of BBR and makes the simple assumption that BBR will usually maintain its $CWND$ at 2 non-congested BDPs – i.e., the BBR gain G is 2. Thus, if the scheduler is initially allowed to use only half of $CWND$, the result should be *full utilization, without much congestion*. The allowable fraction of the $CWND$ is then inflated or deflated based on the scheduler queue occupancy. This approach is *naive*, and will result in performance problems if the gain G is other than the aforementioned size; a larger gain will result in more congestion, whereas a smaller gain will result in under-utilization of the cheaper path. While the gain which ACPFv1 relies on is specific to BBR and CCID5, ACPFv2 is designed to be more independent of the congestion control of the tunnels by instead operating based on an estimation of the minimum RTT – *although we will still use BBR and CCID5 in our implementation for that estimation*. Another issue is the persistence of the BBR bandwidth estimation, which persists through a max-filter for a period of 10 round-trip times. We would like to strive for more adaptive scheduling. The original ACPFv1 scheduler should therefore be thought of as a proof-of-concept only; i.e., because CPF over-schedules packets over the cheap

path it often prohibits capacity aggregation, while ACPFv1 tends to use the second path earlier and thereby avoid congestion *and* increase aggregation.

3.3. ACPFv2 – Low latency packet scheduling for dynamic links

In order to *improve ACPFv1 performance* when scheduling packets over paths with dynamic capacities, e.g. wireless access networks, we have improved the scheduler design. The new version of the scheduler – ACPFv2 – aims to derive the BDP of each path *more precisely* by tracking the *minimum round-trip time*. However, care needs to be taken in order to correctly interpret the state of each path. It is not obvious when a path is underutilized, as the perceived link capacity is capped at the sending rate. Consequently, we allow for 10% more than one BDP in-flight, which will result in a small queue at the bottleneck, as indicated in the right side of Fig. 2. A usable indicator of the utilization of the path is the degree to which the minimum RTT and the current RTT differ. If the current RTT is larger than the minimum RTT , this indicates a queue buildup at the bottleneck buffer, and the path is therefore likely highly utilized. The bigger this difference is, the more sure we may be of full utilization of the path capacity; however, a big difference will – *obviously* – also entail a larger latency. If a queue is drained because of an increase in available path capacity, the scheduler will notice a lower delay and increase the packets scheduled over that path. In our approach, we leverage BBRs capability to estimate the minimum path RTT .

As can be seen in the right-hand side of Fig. 2, ACPFv2 operates in two separate modes, based on the *utilization* of the available paths. As mentioned, ACPFv2 bases its perception of the path utilization on the minimum RTT and the current RTT – *both of which BBR tracks*. A path is considered fully utilized if the current RTT exceeds a target delay, τ , as percentage of the minimum RTT . We use a target delay of 110% of the minimum RTT . As long as at least one path is not fully utilized, ACPFv2 schedules packets according to a modified version of the CPF scheduler, where a fraction of the $CWND$, called the *live $CWND$* , rather than the full $CWND$, is advertised to the scheduler. Henceforth, this mode is called the *underutilized mode*, or mode 1 in Fig. 2. To ensure that a path is not starved, we use a minimum value for the live $CWND$, $cwnd_{min}$, which we set to four packets. Once all paths have achieved full utilization, ACPFv2 switches to the secondary mode, which schedules packets according to the SRTT scheduler. We call this mode the *fully utilized mode*, or mode 2 in Fig. 2. The pseudo code for ACPFv2 is shown in Algorithm 1.

Let $p.mrtt$ denote the current RTT of a path p in the set of paths P , as a percentage of the minimum rtt , denoted as $p.rtt_{min}$. When in *underutilized mode*, the desired change to the current live $CWND$ fraction, $p.frac$, is expressed in terms of $p.mrtt$ and τ . Let inc and dec be the increment or decrement to be made to $p.frac$, where $p.frac$ is bounded by the maximum fraction F_{max} . E denotes the elapsed time since the last adjustment. The adjustment is multiplied by the previous value of $p.frac$, and as a small increment is made after E milliseconds, the adjustment is divided by rtt_{min} in milliseconds. Note, that this does not take into account the compounding effect of repeatedly adding a fraction of $p.frac$ to $p.frac$. The motivation for τ not being 1.0 is that a larger target allows for the discovery of unused capacity; if the capacity is suddenly increased, ACPFv2 will scale up the live $CWND$ fraction by a factor of 1.1 once every rtt_{min} until the target delay, τ , is reached. Although BBR does not have a corresponding target, it would in effect be 2.0 for BBR, as BBR strives to maintain $CWND$ at twice the BDP.

3.4. ACPF version comparison

In order to compare ACPFv1 and ACPFv2 we vary the ACPFv1 $CWND$ fraction and the ACPFv2 delay target, we run a series of experiments where we use both a static 20 Mbps path (blue) and a variable WiFi-like path (red) with an average throughput of ~ 37 Mbps,

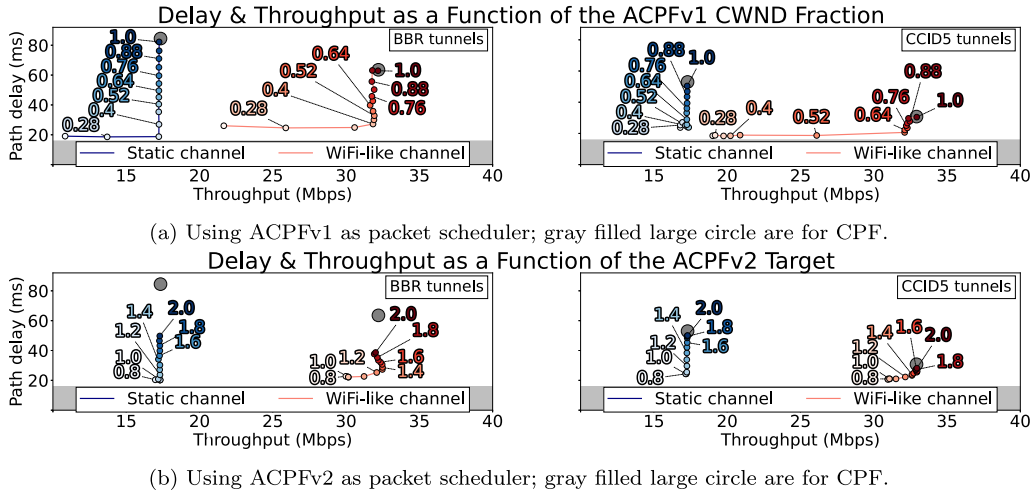


Fig. 3. Impact of the ACPFv1 fraction & the ACPFv2 target on delay & throughput.

Algorithm 1 ACPFv2

```

1: Initialization:
2:    $\tau \leftarrow 1.10$ ,  $cwnd_{min} \leftarrow 4$ ,  $F_{max} \leftarrow 1.00$ 
3:
4: function SCHEDULE( $P, E$ )
5:    $under\_util \leftarrow false$ 
6:    $best\_path \leftarrow null$ 
7:   for  $p \in P$  do
8:      $increased \leftarrow adjust\_cwnd\_frac(p, E)$ 
9:     if  $increased$  then
10:       $under\_util \leftarrow true$ 
11:   if  $under\_util$  then
12:      $best\_path \leftarrow schedule\_acpf(P)$ 
13:   else
14:      $best\_path \leftarrow schedule\_srtt(P)$ 
15:   return  $best\_path$ 
16:
17: function SCHEDULE_ACPF( $P$ )
18:    $min\_prio \leftarrow inf$ 
19:    $best\_path \leftarrow null$ 
20:   for  $p \in P$  do
21:      $live\_cwnd \leftarrow \max(p.cwnd \times p.frac, cwnd_{min})$ 
22:     if  $p.in\_flight \geq live\_cwnd$  then
23:       continue
24:     if  $p.prio < min\_prio$  then
25:        $min\_prio \leftarrow p.prio$ 
26:        $best\_path \leftarrow p$ 
27:   return  $best\_path$ 
28:
29: function ADJUST_CWND_FRAC( $p, E$ )
30:   if  $p.mrtt/p.rtt_{min} < \tau$  then
31:      $inc \leftarrow (\tau/path.mrtt - 1) \times p.frac/p.rtt_{min}$ 
32:      $p.frac \leftarrow \min(E \times inc, F_{max})$ 
33:     return  $true$ 
34:   else
35:      $dec \leftarrow (p.mrtt - \tau)/p.mrtt \times p.frac/p.rtt_{min}$ 
36:      $p.frac \leftarrow E \times dec$ 
37:     return  $false$ 

```

both with a one-way delay of 16 ms. We vary the ACPFv1 CWND fraction in increments of 0.06 while the ACPFv2 delay target is varied in increments of 0.1. We repeat each test 10 times for each parameter combination and plot the average throughput (x-axis) and one-way delay (y-axis) over the primary path. We expect that a low ACPFv2 delay target or ACPFv1 CWND fraction yields low throughput, while a high

ACPFv2 delay target or ACPFv1 CWND fraction results in high delay. The aim of these experiments is to identify which parameter value leads to high throughput while still maintaining a low delay. Fig. 3(a) illustrates the impact of varying the ACPFv1 CWND fraction, for both BBR and CCID5 tunnels. As we can see, increasing the ACPFv1 CWND fraction from a low starting point will increase the throughput, until some point where we reach the maximum throughput, after which increasing the ACPFv1 CWND fraction increases the delay over the tunnel. However, note how the ACPFv1 CWND fraction that archives both the maximal throughput at low delay differs from ~ 0.34 to ~ 0.58 , not only depending on whether BBR or CCID5 is used, but also depending on whether the path has a static or dynamic throughput. In all instances, once the ACPFv1 CWND fraction reaches 1.0 ACPFv1 performs similarly to CPF (gray round circle).

Fig. 3(b) illustrates the impact on ACPFv2 of changing the ACPFv2 delay target. Note that the impact is similar to the impact of changing the fraction for ACPFv1. One notable difference is that ACPFv2 is less prone to under-utilize the primary path. This is because we found it necessary to include a minimum absolute delay difference, which we set to 2 ms. Consider the case where the RTT is 8 ms, and the target RTT is 8.8 ms. As we get the RTT estimation in milliseconds in user-space, ACPFv2 cannot make the necessary distinction between 8.0 and 8.8 ms. However, with the minimum absolute delay difference, the ACPFv2 delay target becomes 10.0 ms instead of 8.8. Another difference is that ACPFv2 performs more similarly regardless of whether BBR or CCID5 is used, particularly in how the ACPFv2 delay target effects the delay. This is because ACPFv2 does *not* regulate its scheduling via assumptions about the BBR/CCID5 gain. We may infer from this that BBR and CCID5 do not use the same gain on average, and that the gain is higher for BBR. This is seen in how the ACPFv2 delay does not reach the CPF delay when BBR is used.

3.5. Competitiveness of ACPFv2

BBR will normally place an entire BDP in the bottleneck buffer, whereas we want ACPFv2 to *initially* limit this to a *smaller amount* so that the scheduler is able to effectively aggregate path capacities. Regardless of the exact queue occupancy we target, it seems clear that this would lead ACPFv2 to *compete poorly* for the cheaper path when it is congested by other flows. There are several reasons why we may assume that this is not as much of an issue as it could be: (i) separate *per-user queues* are common in wireless nodes, which limits the extent to which congestion control algorithms determines the competitiveness of different flows; (ii) there are *multiple paths*, so failing to compete for one path means using another path more extensively; (iii) given

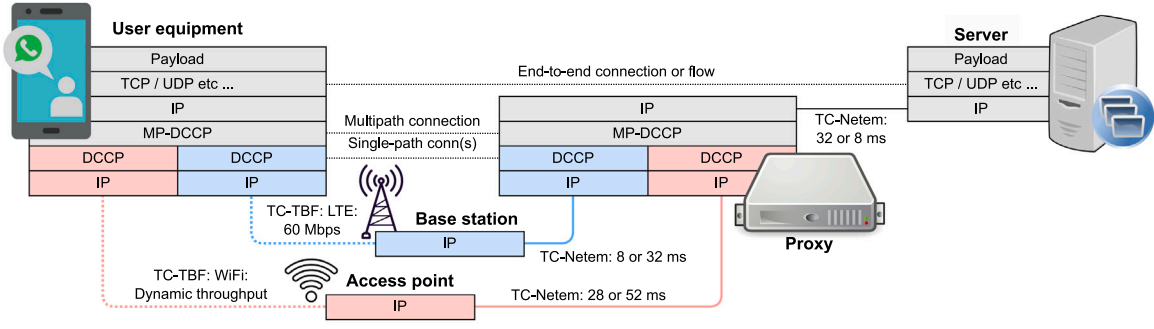


Fig. 4. Network topology/setup used for the evaluation.

the two modes of ACPFv2, the willingness of ACPFv2 to compete for a path *increases* when there is no uncongested path available. Regardless of how big this issue may be, such behavior is still desirable. Since ACPFv2 is designed for non-TCP traffic in ATSSS, which often entails delay-sensitive traffic, the low delay target of ACPFv2 is *non-negotiable*.

If we consider the goal of using cheaper wireless capacity when possible, we may achieve this by using a scheduler that either: (i) utilizes the cheap path *aggressively*, at the cost of an *increased delay*; or else (ii) utilizes the cheap path *less aggressively*, but at a *lower delay*. The former approach is that of CPF, which will use the cheap path more extensively; the latter is the approach of ACPFv2, and it should make the cheap path *usable for a greater variety of flow types*. One might say that ACPFv2 is a complement to CPF for flows that need a lower latency. Using CPF, flows that require low latency may suffer from congestion built up on the cheaper path. The occasional low competitiveness is therefore an intentional feature of ACPFv2.

To elucidate this behavior, assume that a flow scheduled by ACPFv2 starts when the cheap path is *already congested* by some other flow. If there is a *per-flow queue* at the cheap path bottleneck, ACPFv2 should correctly sense the minimum RTT, the flow should get its fair share of that bottleneck, and ACPFv2 starts offloading onto the expensive path once it sees an increase in the current RTT over the minimum RTT. If, however, there is only a *single shared queue*, the result will be different. Ideally, ACPFv2 would correctly observe the minimum RTT, identify the existing congestion, and consequently shift more traffic to the expensive path. Note that this is by design because with ACPFv2 we aim to support flows that require a low latency service from the network. Conceivably, ACPFv2 may result in sub-optimal performance in this case if it conflates a congested RTT for the minimum RTT, and therefore schedules packets over a congested path when it should not. One way to cope with such a situation would be to place an absolute delay limit on a path that would trigger ACPFv2 to view the path as unusable or use a more reliable method of estimating the minimum RTT, which we consider future work. Note that the latter is *not possible* to do for ACPFv1, as it does not regulate its behavior based on the minimum RTT and current RTT.

4. Implementation

We have implemented ACPFv2 both in a user-space framework, as well as integrated the scheduler into the kernel-space MP-DCCP protocol using kernel version 4.14.¹ Although implementing ACPFv2 in the two environments requires slightly different approaches, both versions are functionally very similar. Our implementations of ACPFv2 use a separate management function (`adjust_cwnd_frac` in Algorithm 1), which updates the live CWND used to schedule packets. While in the user space version this function is called periodically using a separate thread ($E = 1$ ms), the kernel-space version calls this function just

before a scheduling decision is to be made (on the same thread). Kernel-space ACPFv2 therefore does not assume that `adjust_cwnd_frac` is called periodically, and instead scales the increment/decrement by E ms, i.e. the time that has elapsed since the last adjustment, with a minimum of one millisecond between adjustments.

5. Evaluation setup

In order to evaluate ACPFv2, which we will refer to simply as ACPF from here on out, we use the Mininet network emulator with both kernel and user-space implementations of the scheduler. We first describe the multipath frameworks we use in Section 5.1; next we cover the network topology in Section 5.2; we show how WiFi links are emulated to create realistic scenarios in Section 5.3; finally, we detail our traffic generation in Section 5.4.

5.1. Multipath framework

We have implemented ACPF in the Linux kernel (4.14) version of MP-DCCP [6] as well as in a protocol agnostic user-space multipath tunneling framework, which both include CPF and SRTT schedulers. Our multipath tunnel spans from the *user equipment* to the *proxy*, as illustrated in Fig. 4. The tunnel receiver buffers packets to restore the original packet order, but limits the buffer time based on the path delay asymmetry. The user-space framework implements all scheduling and reordering logic in user-space, but uses kernel-space protocols for sending packets over individual paths. This approach allows us to use both per path DCCP and TCP tunnels in our evaluation and study the benefit of using multiple unreliable tunnels together with ACPF for non-TCP traffic.

5.2. Topology

For our evaluation, we deploy the network topology from Fig. 4 in Mininet [30]. We use TC-Netem [31] to emulate propagation delay in the core network and over the Internet, while TC-TBF [32] is used to emulate wireless links. We run several experiments to determine typical RTT values to different regional servers over cellular and over WiFi, using the ping utility. With the RTT values established, to evaluate the impact of the proxy location, we create two deployment scenarios; the (i) *cloud scenario* has the path RTTs at 32 and 52 ms, while the (ii) *edge scenario* have the path RTTs at 8 and 28 ms; we also use an (iii) *asymmetric scenario* based on the edge scenario but with the delay asymmetry increased by a factor of 4 over the asymmetry of the edge and cloud scenarios, as well as a (iv) *reversed asymmetric scenario* where the path delays are reversed compared to scenario (iii). Regardless of deployment, we set the *end-to-end* round-trip time (RTT) to approximate the previously mentioned observed RTT values of 40 ms over the primary WiFi path, and 60 ms over the secondary cellular path, except for the two asymmetric scenarios where one path is much longer by necessity. Note, that we assume the RTTs are similar to what

¹ https://github.com/telekom/mp-dccp/tree/mpdccp_v03_k4.14

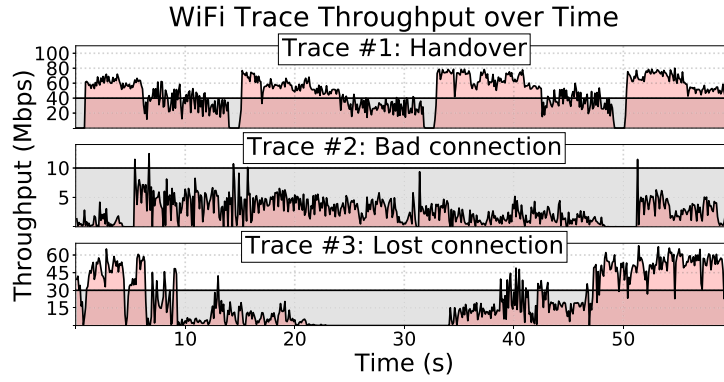


Fig. 5. WiFi throughput traces; gray indicates UDP & SRT application sending rates.

Table 1

Parameters used in the evaluation. The primary path corresponds with the WiFi access, and the secondary path with the cellular access.

Scenario	Path RTT values				
	Internet	Primary	Secondary	E2E over primary	E2E over secondary
Edge deployment	32 ms	8 ms	28 ms	40 ms	60 ms
Cloud deployment	8 ms	32 ms	52 ms	40 ms	60 ms
Asymmetric	32 ms	8 ms	88 ms	40 ms	120 ms
Reversed asymmetric	32 ms	88 ms	8 ms	120 ms	40 ms
		Parameter			
		Value			
		Scheduler buffer size			
		1000 pkts			
		Bottleneck buffer size (both paths)			
		500 pkts			
		Primary path throughput			
		Dynamic WiFi-like			
		Secondary path throughput			
		60 Mbps, or dynamic LTE-like			
		WiFi & cellular path loss			
		0% or 1%			

we measured above, despite adding a proxy to the end-to-end path. Therefore, we implicitly assume that in our scenarios a proxy would not impose a significant detour compared to the end-to-end flow. Table 1 shows the full set of parameters.

5.3. Wireless link emulation

To identify typical WiFi link behavior scenarios, we collect real traces by saturating a WiFi channel with UDP Iperf traffic while walking around our university building and parking lot. We calculate achievable throughput over time at a 100 ms granularity. The collected traces are then used as input for a python agent that issues TC-TBF commands at the same granularity in order to emulate a WiFi channel. We created three different WiFi mobility scenarios (see Fig. 5): a (i) *handover* scenario where we walk inside a building creating several handovers with short disconnections in between; a (ii) *bad connection* scenario, where we walked around outside close to the building with a generally poor WiFi channel; and a (iii) *lost connection* scenario, where we walked towards the parking place showing a longer WiFi outage. We assume a static LTE channel. The motivation for this is that ACPF prefers the path containing the WiFi link, and the cellular path only serves as a *temporarily stand-in* whenever the WiFi capacity is insufficient. However, we do run a few tests where the LTE channel quality varies. As a default, both paths are given a loss rate of 0%, except for outages which are emulated with a loss rate of 100%. We also consider the case when both paths have a loss rate of 1%.

It is reasonable to assume that the *propagation delay* was static during the gathering of our traces, with most of the variability in delay being driven by network congestion. The propagation delay will change between access point handovers, such as those in the handover scenario, but we omit dealing with this in our evaluation. The back-off behavior of the WiFi MAC-layer should be a relatively minor source of the delay, with the possible exception of the bad connection scenario. Nonetheless, we also evaluate ACPF in the user-space framework in the presence of RTT noise *similar* to what might be expected from a

WiFi MAC-layer with an exponentially increasing back-off timer. We calculate that the *worst-case* back-off wait time is roughly 20 ms, and emulate this *added delay* using TC-Netem by adding the back-off wait time to the propagation delay, and reverting to the propagation delay after a period has passed equal to the back-off wait time. Since TC-Netem is somewhat crude, we only emulate one such back-off event per 100 ms, and we scale the wait time between 0 and 20 ms depending on the current link capacity relative to the peak capacity.

5.4. Traffic generation

Our primary interest is to evaluate the performance of ACPF for unreliable traffic. We focus in this paper on stream-like flows and we generate end-to-end traffic using UDP, SRT, YouTube or WebRTC. The former two are evaluated in our user-space framework, while the latter two are evaluated using the Linux kernel MP-DCCP implementation.

5.4.1. UDP and SRT traffic

We use Iperf to generate UDP traffic, at rates of 40 Mbps, 10 Mbps, and 30 Mbps for the three WiFi scenarios in Fig. 5, respectively. Given these rates, aggregation will only be continuously required for the *bad connection* scenario. *Secure reliable transport* [33] (SRT) is designed for both sub-second video streaming as well as file transfers. We run SRT in streaming mode with the same data rates as those given to UDP Iperf, using a publicly available traffic generation tool.² We set the sender and receiver buffer sizes to 1 GB each; although ostentatious, our testing indicates that this size has no impact on performance relative to other sizes. Otherwise, we keep the default settings; including the maximum accepted transmission delay of 120 ms, and the connect timeout of 3000 ms.

² <https://github.com/maxsharabayko/srt-xtransmit>

5.4.2. WebRTC traffic

To generate WebRTC traffic, we set up a one-way video stream from the server to the UE client, and collect statistics using the `getStats()` API. We use a Node.js library called Puppeteer³ which is able to control Chromium based browsers such as Google Chrome programmatically. We run the browser *headlessly* in order to minimize the impact on CPU and GPU resources. This ensures that network performance becomes the deciding factor in the quality of the playback. We then instruct the browser on both the UE and server to load a web page that runs the WebRTC application and collect the statistics from `getStats()`.

5.4.3. YouTube traffic

To generate YouTube video traffic, we let a web browser play a video over the YouTube video streaming platform. Here, we also use Puppeteer in order to instruct a web browser on the UE to fetch a web page with an embedded YouTube player. The web page uses the YouTube Iframe API⁴ to monitor the status of the player through Javascript. In addition, a competing greedy Iperf TCP flow is established over the primary path.

6. Evaluation results

By design, ACPF should maintain low queuing delay while being able to fully utilize the primary path capacity. In our evaluation, we answer the following questions:

1. To what extent does ACPF reduce the bottleneck queuing delay over the primary path?
2. To what extent does ACPF impact the utilization of the primary path?
3. How does the proxy deployment and path characteristics impact performance parameters?
4. How does ACPF impact the end-to-end quality of experience (QoE) for different streaming applications?
5. How does ACPF perform in the presence of competing flows?

To answer these questions, we compare ACPF with CPF and SRTT for different traffic and deployment scenarios, using both user-space and kernel-space implementations of these three schedulers.

We first show an illustrative example of achievable per tunnel throughput and delay using the handover scenario from Fig. 5, with end-to-end UDP traffic at a rate of 40 Mbps over our ATSSS framework. Fig. 6 shows the performance when using CPF (top) or ACPF (bottom). The behavior of path 1 (WiFi) is plotted in different shades of red — as opposed to shades of blue for path 2. Capacity aggregation is not always required, as the 40 Mbps sending rate can often be sustained on the primary path. While a *low delay* and a *high throughput* over the primary path is desirable, we see that the primary path delay remains near the 16 ms propagation delay, but that spikes of up-to 80 ms occur frequently during aggregation periods. These spikes are much lower when ACPF is used, while the throughput over the primary path is *seemingly unchanged*.

In the remainder of this chapter, we analyze in more detail the performance characteristics of ACPF. In Section 6.1, we analyze the *fundamental scheduling behavior* of ACPF, and demonstrate that ACPF reduces queuing delay while keeping primary path utilization high. In Section 6.2 we evaluate ACPF performance while varying various path characteristics. In Section 6.3, we evaluate the impact of competing flows and congestion on ACPF. In Section 6.4 we focus on the *end-to-end application QoE*, and demonstrate that ACPF achieves a *higher video resolution* and reduces the risk of prematurely terminating a streaming session.

6.1. Fundamental scheduling behavior

In this section, we aim to answer the first two questions posed previously, namely how ACPF impacts the bottleneck queuing delay of the primary path and what is the impact on path utilization. We mostly refer the reader to the full set of results from Table 2. Performance numbers presented below are *average values* over different scenarios from the Table 2, where we describe the scenarios selected in greater detail below.

ACPF results in lower queuing delay than CPF or SRTT. When comparing the queuing delay achieved by CPF and ACPF, we distinguish between using CCID5 or BBR tunnels. With DCCP tunnels, ACPF reduces delay *relative to CPF* by 16%–76% or 0.5–33 ms across all our scenarios, with the average being 46% or 11 ms; note that ACPF produces less delay than CPF *in all scenarios in Table 2*. As the WiFi capacity often exceeds the application sending rate, the queuing delay is often *near zero*. However, the delay spikes up-to 80 ms when the WiFi link becomes insufficient. This variability is seen in Fig. 7, which depicts the cumulative distribution function of the queuing delay. Fig. 7(b) in particular shows that while ACPF outperforms CPF, the delay could be further improved for the weak WiFi scenario, where *aggregation is continuously required*. Fig. 8(a) focuses on SRT results, and shows lower delay when using ACPF. Using UDP results in similar performance trends, as can be seen in Table 2. Finally, per Table 2, with TCP tunnels ACPF reduces delay *relative to CPF* by 6–94% or by 2–257 ms, which averages to 65% or 63 ms. Fig. 9 visualizes all CPF and ACPF test cases from Table 2 in a scatter plot to illustrate how ACPF reduces the queuing delay compared to CPF. Note the general leftward shift towards lower delay for ACPF (blue) compared to CPF (red).

ACPF maintains high cheap path utilization. When comparing the traffic distribution between the primary and secondary path for CPF and ACPF, we separate scenarios without competing traffic from scenarios with competing traffic. Averaged over *all scenarios in Table 2*, except those with competing traffic, ACPF shifts the traffic distribution relative to CPF 3.9 percentage points towards the expensive path, or 4.0 percentage points when excluding SRT tests — *some of which terminates prematurely*. There were no competing flows over the bottleneck for these tests, and the aggregate capacity was enough to satisfy user demands. Consequently, a high utilization of the cheap path is desirable. However, the slightly less aggressive usage of the cheap path we do observe enables ACPF to significantly reduce the latency — *per Table 2*. Fig. 8(b) focuses on the SRT results, and shows how ACPF schedules *slightly* more traffic on the secondary path compared to when using CPF. The UDP results are very similar, as can be seen from Table 2. With competing traffic, ACPF increases the secondary path usage by 6.6 percentage points. As ACPF starts to shift traffic earlier to the secondary path in order to keep the delay lower, more traffic is sent over the secondary path. This is also visible from the scatter plot in Fig. 9, where ACPF results are consistently shifted downwards, indicating more traffic is sent on the secondary path. Table 3 summarizes results obtained from the kernel-space MP-DCCP tunneling framework, and show that for the WebRTC tests the percentage over the cheap path is 74% for ACPF, 84% for CPF, and 78% for SRTT.

6.2. Impact of path characteristics

In this section, we attempt to identify the impact of the proxy location and various path characteristics on the ACPF performance by analyzing results from Table 2.

ACPF uses the cheap path less if there is delay noise. When looking at the impact of MAC-layer delay noise on ACPF, we compare the ACPF performance with and without the added delay noise both for CCID5 and BBR tunnels, separately. Averaged across all DCCP-tunnel scenarios in Table 2 the average traffic distribution over the cheap path is 57% *without* and 53% *with* added delay noise; the average queuing

³ <https://pptr.dev/>

⁴ https://developers.google.com/youtube/iframe_api_reference

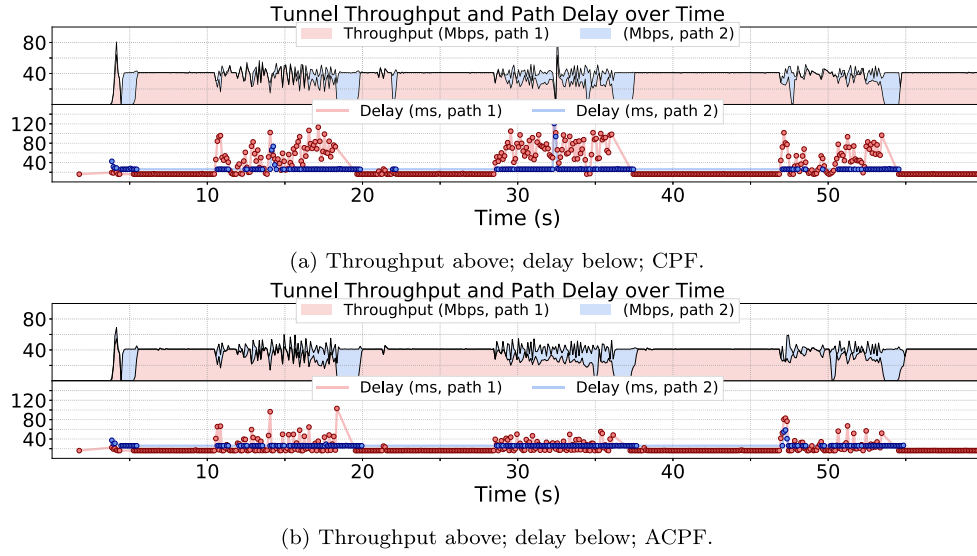


Fig. 6. Example results; CPF (top) & ACPF (bottom); UDP end-to-end.

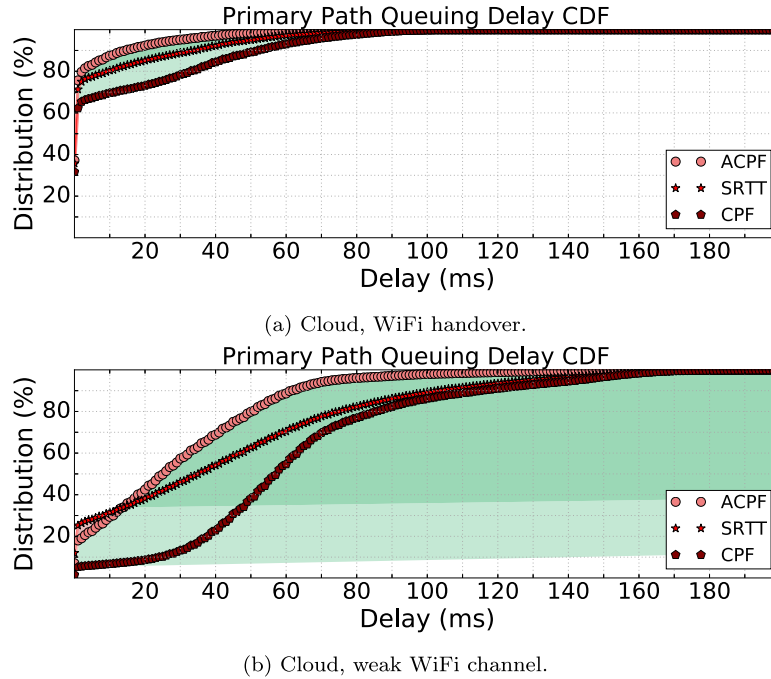


Fig. 7. Path 1 queuing delay CDF for all schedulers & two WiFi scenarios; UDP E2E.

delay is 10 ms *without* and 9 ms *with* added delay noise. However, when using TCP for the tunnel, without delay noise the utilization is around 58%, while with added noise the utilization drops to 52%, while the delay change from 18 ms to 17 ms, respectively. The utilization change is not surprising, as it is difficult for ACPF to distinguish between congestion delay and other transient delay sources. ACPF may accidentally mistake transient MAC-layer delay as congestion delay, and therefore erroneously infer that the path is fully used. That the delay is actually reduced despite there being an added delay source is also not surprising; ACPF is simply overcompensating for the observed delay spikes.

ACPF performs better with a short tunnel delay. Our scenarios include three different primary path RTT values. If we sort our scenarios by decreasing primary path RTT and look at Table 2, we see that the average queuing delay is 28 ms for the reversed asymmetric scenario, 14 ms for the cloud deployment, and 6 ms for the core deployment

and asymmetric scenarios, while the traffic distribution over the cheap path is 61%, 69%, and 73%, respectively. Note, that these numbers only include tests using the handover Wifi trace, which allows for a comparison that includes the asymmetric scenarios. However, similar performance trends were visible for the other two traces. For the WebRTC results, we note that the traffic over the cheap path is 71% for the cloud deployment and 77% for the edge deployment. This improvement is due to the much shorter congestion control loop for the CCID5 tunnels when the proxy is deployed in the edge, which enables faster adaptation to the variable capacity of the path.

Large delay asymmetry does not effect ACPF scheduling efficacy. When using CCID5 tunnels, the average primary path queuing delay for ACPF amounts to 2 ms for the *edge scenario*, and 2 ms for the *asymmetric scenario*. When using BBR tunnels, these numbers change to 9 ms and 11 ms, respectively. These values are the results from columns E and EB in Table 2. The queuing delay values of columns

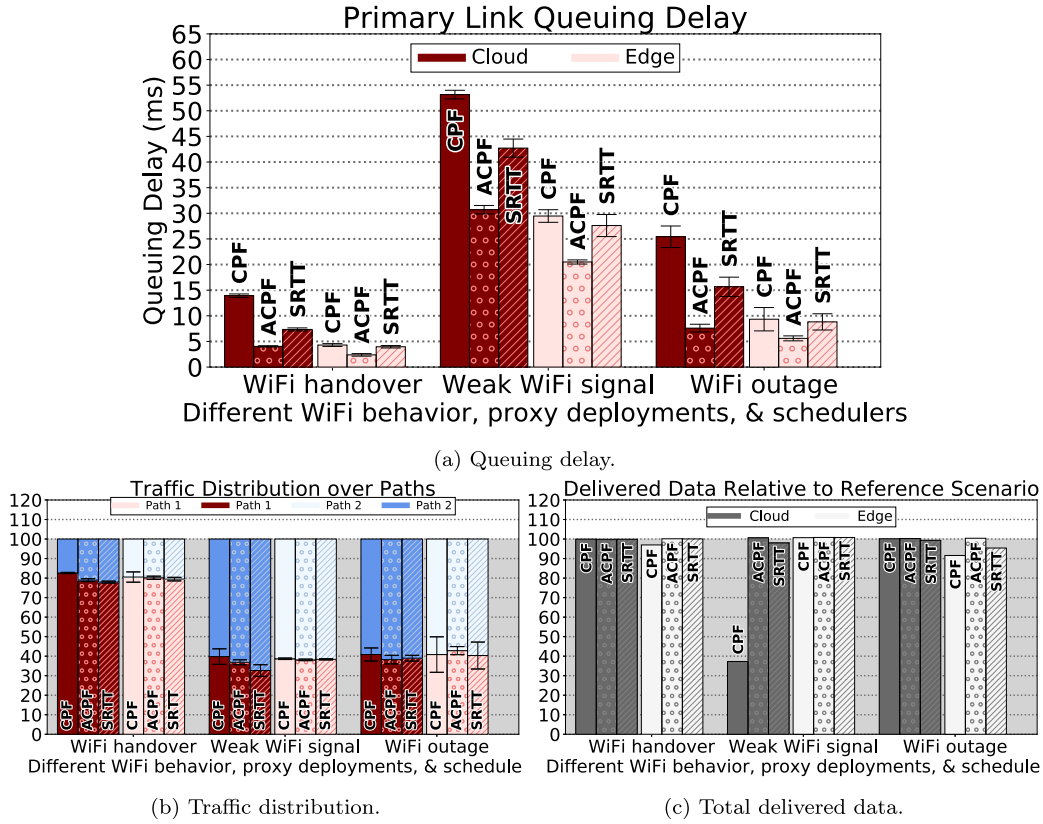


Fig. 8. Results for SRT E2E; DCCP-CCID5 tunnels; for all schedulers & scenarios.

Table 2

All user-space results. Each value is the averaged over 10 repetitions. For calculating the queuing delay, we subtract the propagation delay from the total delay over the path. The delivered data is normalized against a reference scenario where no aggregation is required (the cheap path had sufficient capacity). The traffic distribution is calculated by dividing the delivered data over the path by the total amount delivered over both paths, for that scenario. UDP & SRT refers to the traffic type; DCCP-CCID5 & TPC-BBR refers to the tunnel protocol. Blue indicates performance improvements of ACPF, while red indicates performance loss. For improvement/loss, we require a difference of at least 2 ms delay, a delivered data difference of 5%, traffic distribution difference of 2%. This color aid is always based on comparing CPF to ACPF. The total delivered data will vary for SRT because of its circuit breaker, and because of SRT re-transmissions. Rows 1, 13 & 25 are also plotted in Fig. 8. Rows 1 to 12 & 25 to 36 are also plotted in Fig. 9.

			Handover scenario												Bad connection scenario												Lost connection scenario											
			No competing traffic						Competing UDP traffic						No competing traffic						No competing traffic						No competing traffic											
			Cloud deployment			Edge deployment			Cloud deployment			Edge deployment			Very long 2 nd path			Very long 1 st path			Cloud deployment			Edge deployment			Cloud deployment			Edge deployment								
			CPF	ACPF	SRTT	CPF	ACPF	SRTT	CPF	ACPF	SRTT	CPF	ACPF	SRTT	CPF	ACPF	SRTT	CPF	ACPF	SRTT	CPF	ACPF	SRTT	CPF	ACPF	SRTT	CPF	ACPF	SRTT	CPF	ACPF	SRTT						
Queuing delay (ms)	SRT	CCID5	Static delay	13.9	4	7.3	4.3	2.4	4	16.6	5.8	7.6	5.5	1.9	3.8	4.3	2.3	3.9	32.7	9.9	3.1	53.2	30.7	42.7	29.5	20.5	27.6	25.4	7.6	15.7	9.3	5.6	8.8	1				
			MAC emu.	12.3	3.9	7.8	3.5	2.9	3.2	15.8	5.2	7.9	7.4	2.3	3.7	3.1	2.4	3.1	33.5	8.3	12.4	55.6	27.9	43.1	24.1	15.6	21.9	24.1	7.5	17.6	8.2	5.4	7.2	2				
			MAC emu. & loss	13.5	4.5	7.8	3.6	2.1	3.7	18.5	5.2	7.9	5.5	2.3	3.8	4.6	3.1	4.9	26.6	9.5	40	50	27.8	36.7	25	14.4	22	34.5	10.6	20.3	11.6	6.6	10.9	3				
			Static delay	68.1	9.6	20.4	22.3	5.8	13	55.7	8.3	12.7	22.3	13.8	7.6	20.5	3.3	17.9	81.6	17.3	96.5	161	47.1	91.6	61.7	21.6	66	271.1	29.6	88.8	89.6	10.3	43.5	4				
	BBR	CCID5	MAC emu.	68	8.7	23	19.6	7.4	13.9	97.3	8.1	13.8	18.5	3.9	8.4	17.9	4.2	17	89.4	11.1	35.5	173.6	47.8	101.4	77.3	23.6	62.4	284.6	28	100.6	96.4	12.4	49	5				
			MAC emu. & loss	80.9	57.1	66.8	21.7	14.5	19.2	81.7	44.4	49.3	38	16.5	20.1	28.6	2.7	30.1	149.8	101	120.2	146.9	86.5	133.7	76.9	32.8	71.1	293.9	223.3	217	75.1	60.1	89	6				
			Static delay	13.9	4.2	7.7	4	2.3	3.8	16.5	5.5	6.8	5.2	1.7	3.8	3.9	1.9	3.9	34.8	10.2	8.6	62.7	29.4	42.8	28.8	19.7	27	25.8	7.3	15.4	8.1	5.4	8.1	7				
			MAC emu.	11.6	3.6	7.9	3.1	2.5	3.3	15.4	4.3	7.7	5.1	2.2	3.6	3.7	2.3	3.4	37.5	9	13.3	57.4	25.3	43	24	14.9	20.4	23.5	5.9	15.5	8.3	4.9	6.4	8				
	UDP	CCID5	MAC emu. & loss	11.9	4	6.8	3.2	2	3	15.4	4.3	7.7	5.1	2.2	3.7	3.3	2.5	3.4	36.5	9	19.2	56.1	25.9	35.2	24.8	14.2	21	24.6	6.7	14.2	7.6	4.4	6.8	9				
			Static delay	66.5	10	23.1	23.7	5.6	14.1	106.2	9.1	12.2	63	4	9.8	24.2	4.5	20.8	101.5	21.1	54	187.6	47.4	114.4	123.3	21	81.2	212.4	27.1	83.8	58	9.4	41.4	10				
			MAC emu.	65.2	8.9	22	21.6	6.7	16.3	112.4	9.4	15.1	58.5	4.4	9.2	21.3	5.1	20.6	122.7	15.7	49.3	178.7	43.9	130	118.9	20.8	82.1	207.5	30.7	107.7	57.2	11.1	48	11				
			MAC emu. & loss	60.6	48.6	52.5	22.6	16	19.7	76.3	44	45.8	34.2	17.3	21.8	25.7	19.8	24.7	157.2	107.7	117.5	182.1	91.2	139.5	117.2	31.1	90.5	236.8	193.6	223.4	80.1	71.6	75.6	12				
Delivered data (%)	SRT	CCID5	Static delay	100	99.9	99.9	97	100	100	100	99.9	99.9	99.7	100	100	100	99.9	99.9	100.0	100.6	96.6	37.3	100.6	98	100.7	100.8	100.8	100.1	100.3	99.3	91.6	100.4	95.4	13				
			MAC emu.	99.9	99.8	99.8	100	100	100	99.9	100.1	99.7	99.7	100	99.9	99.3	99.8	99.6	100.8	101.1	101.5	72.7	100.5	100.5	100.8	100.7	100.7	94.6	91.9	95.4	100.4	100.4	99.7	14				
			MAC emu. & loss	70.2	100.6	92.4	102.2	101.8	101.4	72	101.6	102.5	101.5	102.2	101.8	36.3	34.6	36.3	105.7	115.1	440.4	36.7	89	70.3	88.8	101.7	102	35.6	47.6	38.3	42.9	71.4	47.5	15				
			Static delay	76.7	100.1	100.5	100.1	100.1	100.3	46.7	98.8	100.1	100	99.9	100	100.3	104.4	104.6	38.9	71.8	100.4	30.9	98.4	69.9	39.2	101.1	59.8	42.1	101	101	54.5	100.4	98.6	16				
	BBR	CCID5	MAC emu.	77.9	100.2	100.6	100.3	100.2	100.3	64.3	100.1	100.1	100.1	100	100	100.2	99.8	100.2	45.4	100.2	100.5	33.6	101.4	72.6	46.1	101.2	79.9	43.8	101	101.3	64.5	100.4	100.8	17				
			MAC emu. & loss	60.1	106.1	110	105.4	103.3	104.8	100.2	107.6	108.1	72.9	102.2	103.2	58.5	40.6	67.3	81.5	76.3	77.9	86.4	107	109.5	57.4	102.8	92.7	54.9	106.3	107.9	43.5	103.1	81.4	18				
			Static delay	99.9	100.1	99.7	100.1	100	100	99.4	99.7	99.8	99.9	99.9	99.9	98.1	99.2	98.9	98.7	99.6	100.2	97.7	99.3	97.9	99.7	99.9	99.6	99.7	100	99.8	100.2	100.1	98.6	19				
			MAC emu.	100.1	100.2	99.9	100	100	100	99.8	99.8	99.9	99.9	99.9	99.9	98.4	97.9	98.5	99.6	100.9	101.4	97.9	98.2	98.4	99.4	99.4	99.4	99.9	100	99.1	99.8	99.9	98.3	21				
	UDP	CCID5	Static delay	100	100	100	100	100	100	100	100	100	100	100	100	100	100.2	100.3	100.3	100	100	100	100.2	100.1	100.1	100.2	100.2	100.6	100.1	100.1	100.1	100.1	100.1	100.2	22			
			MAC emu.	100	100	100	100	100	100	100	100	100	100	100	100	100	100	99.3	100	100.4	100.3	100.4	100	100.2	100.1	100.1	100.2	100.1	100.6	100.1	100.1	100.2	100.1	100.2	23			
			MAC emu. & loss	100	98.5	100.1	99.1	100	100.1	100.3	100.3	100.4	99.7	100	99.9	100	100	100.6	100.4	100.6	100	100	100.2	100.3	100.3	100	100	99.8	100.1	100.1	100.1	99.8	99.2	24				
			Static delay	82.6	78.9	77.9	80.7	80.1	79.5	74.7	70.7	70.1	71.9	69.4	71.9	81.8	80.9	81.5	77.6	76.6	1	39.8	36.9	32.6	38.7	38.1	38.4	40.8	38.2	39	40.7	42.8	40.3	25				
Traffic distribution (%)	SRT	CCID5	MAC emu.	82.2	68.8	73.1	77.7	76.4	73.9	73.6	64.9	67.1	69.3	64.2	66.4	78.2	74.6	77.6	78	76.3	0.9	39.8	35.8	30.5	38.6	37.5	35.1	40.6	32.1	38.1	40.9	39.8	39.2	26				
			MAC emu. & loss	79.7	66.7	74	76.8	68.2	73.6	70.9	63.5	65.2	69.1	64.1	67.2	72.9	65.8	74	76.1	54	2.7	34.2	38.2	33.2	39.3	37	34.5	51.2	33.6	44.3	45.7	32.4	38.2	27				
			Static delay	78.6	79.2	76.1	81.3	79.2	76.6	79.2	71.5	71.5	75	68.9	72.8	81.4	78.7	81.3	72.8	71	0.7	33.8	38	36.2	44.4	38.1	42.5	48.9	43.5	39.3	38.1	44.3	40.2	28				
			MAC emu.	78.4	66.5	71.6	81.2	75.7	74.6	72.2	64.5	67.4	74.4	68	69	81.3	66.5	81.1	74.2	57.1	0.6	37.2	36.5	34.9	45	37.5	37	46.2	34.7	36.2	32	41	38.5	29				
	BBR	CCID5	MAC emu. & loss	54.8	49.8	52.6	62.3	58.7	61.6	55.1	52.5	52.2	57.8	58	60.3	64.4	56.8	63.4	42.3	33.5	17	37.2	32.7	29.9	42.4	36.2	36.1	31.8	27.7	31.1	40.9	34.4	31.2	30				
			Static delay	82.9	79.8	78.3	81.1	81	80.3	75.3	71.7	69.9	72.6	70.9	71.4	80.6	79.6	80.8	77.4	75.7	0.8	38.4	37.3	33.8	39.6	39.1	39.7	42	36.1	39.5	43.4	42.2	42.5	31				
			MAC emu.	82.5	67.7	73.8	77.9	76.3	72.5	75.1	64.2	68.4	69.3	64.1	67.1	77.7	73.7	78.1	77.6	64.3	0.7	36.7	36.3	30.6	39.6	38.5	30.7	43.6	32.7	37.4	43	40.2	35.1	32				
			MAC emu. & loss	81.7	67.4	74.9	77.6	72.2	72.8	74.3	64.7	66.7	69.9	64.1	67.4	77.9	74	78.3	77.3	64.8	0.7	35.5	35.4	30.3	37.7	36.7	35	43.9	34.3	37.5	41.2	35.7	40	33				
	UDP	CCID5	Static delay	79.7	79.7	74.9	81.9	80.4	75.9	76.5	70.6	70.5	75.3	68.2	71.2	80.8	79.3	80.9	73.8	71.3	0.6	38.2	39.1	33.9	41.3	39.7	38.9	45	42.2	41.7	47	44.2	42.7	34				
MAC emu.			79.6	67.4	71.1	81.2	78.2	78.1	75.9	63.8	66.5	73.5	69.3	69	80.5	65.5	79.9	72	58.1	0.6	38	37.8	32	40.9	39	35.5	45.9	35	36.9	46	42	40.2	35					
MAC emu. & loss			57.3	50	54.2	62.6	58.3	61.3	57	51.5	53.9	61.9	57.8	59.9	66.2	62	66.6	48.5	35.5	17	35.6	33.7	32.4	37.4	36.4	35.7	33.5	30.2	29.1	35.8	33.3	34.9	36					
A			B	C	D	E	F	A	B	C	D	E	F	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R							

Table 3

The distribution of traffic (%) that were sent on the primary path, using the three different schedulers. From the kernel-space experiments.

	Handover scenario						Bad connection scenario						Lost connection scenario					
	Cloud deployment			Edge deployment			Cloud deployment			Edge deployment			Cloud deployment			Edge deployment		
	CPF	ACPF	SRTT	CPF	ACPF	SRTT	CPF	ACPF	SRTT	CPF	ACPF	SRTT	CPF	ACPF	SRTT	CPF	ACPF	SRTT
Youtube	34.0	13.8	13.8	33.1	13.0	11.6	11.9	2.1	2.0	4.9	1.3	1.4	26.9	10.0	11.1	26.5	11.0	11.1
WebRTC	94.6	89.5	87.5	93.1	89.7	91.7	88.1	60.2	80.5	88.8	70.2	76.1	68.3	62.6	60.8	72.9	70.8	69.1

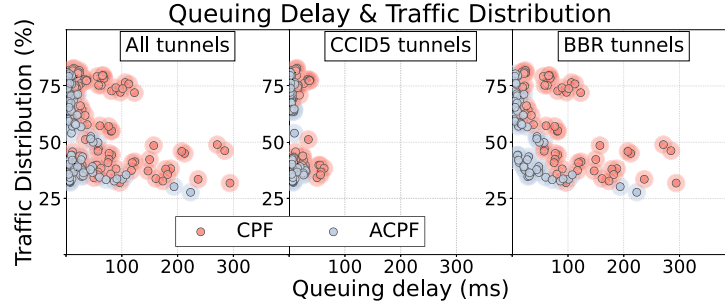


Fig. 9. Comparison of CPF & ACPF across all scenarios, excluding those using SRTT.

does not re-transmit packets, or buffer packets in order to restore the correct packet order. Moreover, the delay is measured from *socket to socket*, and so we do not capture buffering that might happen at the multipath level at the receiver.

Delay is not affected by path loss if the tunnel is unreliable. When adding *path loss*, the average primary path queuing delay across all scenarios with ACPF when using CCID5 tunnels is increased by a mere 0.3 ms, from 8.9 ms to 9.2 ms. When using BBR tunnels instead, the increase is much more significant, from 18 ms to 74 ms. That BBR tunnels should exhibit a longer delay in the presence of loss is natural, as BBR – or rather TCP – withholds the release of packets pending the arrival of re-transmissions. Comparing ACPF to CPF over BBR tunnels, we see that when there is no loss, ACPF decreases the delay by 84 ms on average. The presence of loss changes this decrease to 36 ms on average. We thus conclude that implementing reliability by TCP style re-transmissions over a path with packet loss *partially undoes* the performance gains achieved by ACPF.

6.3. Congestion & competitiveness

In this section, we aim to analyze the impact of congestion on achievable performance by analyzing results from Table 2. However, we also present a few scenarios that are not in this table.

ACPF competes less for a congested path when an uncongested path is available. For columns AA to FA in Table 2 we use an additional 20 Mbps constant bit-rate UDP flow going over the primary path, outside of the multipath tunnel. When using ACPF with CCID5 tunnels, the queueing delay increase marginally from 3 ms to 4 ms when adding the competing flow. For BBR, the delay increase from 15 to 17 ms. Adding the competing flow changes the fraction of the packets going over the primary path from 73% to 66% when using CCID5 tunnels, and from 68% to 62% when using BBR tunnels. This is expected as ACPF is forced to reduce its utilization of the primary path to maintain a low delay. However, results change when we use a competing TCP flow instead of a CBR UDP flow. Our YouTube tests evaluate our ACPF scheduler in the presence of an additional competing TCP flow over the cheap path. Looking more closely at Table 3 we see that the utilization of the cheap path is at 8.5% for both ACPF and SRTT averaged over all three scenarios, while being at 23% for CPF. Note that we *actually want the lower utilization of the cheap path* in this case, since competing for this path would entail increasing the end-to-end delay.

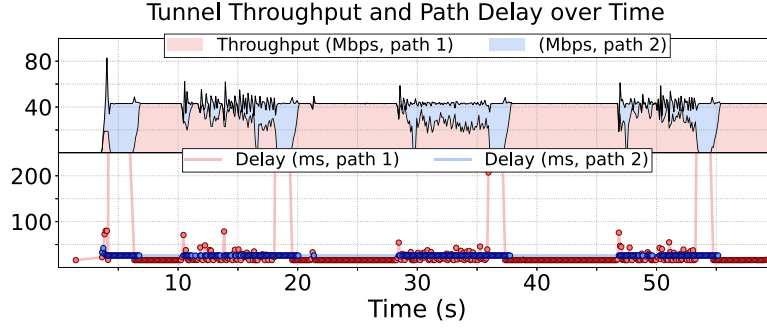
ACPF is unwilling to completely abandon a heavily congested path. When there is a competing flow on the primary path that is

willing to capture the *entire* throughput, ACPF is unwilling to back off the primary path *as much* as one would like. If low delay is desired, and the secondary path is not much longer than the primary path and the secondary path is able to satisfy the end-to-end flow, ACPF should ideally switch completely to the secondary path, rather than competing for the highly congested primary path. If the secondary path is much longer than the primary path, ACPF should compete harder for the primary path given that its secondary mode – *i.e., only congested paths are available* – is to schedule like SRTT. Note, that tests using competing greedy flows are not part of Table 2, as we deemed this issue to be mostly out-of-scope, and consequently did not run a full suite of these tests. We omit any figures that illustrate this issue. However, when we do run a test using the cloud deployment, BBR tunnels, and greedy competing traffic the outcome is a queuing delay over the primary path of 46 ms and 33% of the ACPF flow going over the cheap path. Note that the issue here is that the amount of traffic going over the cheap path should be much lower.

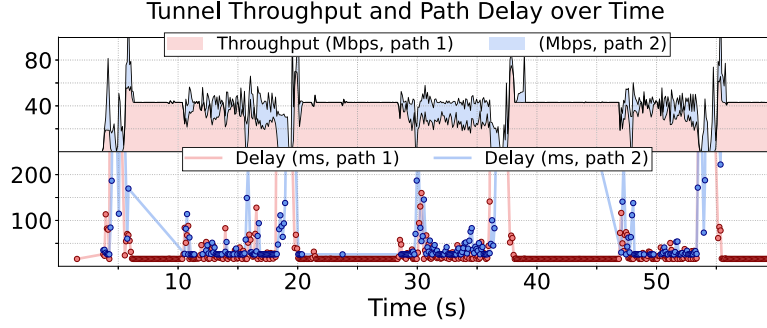
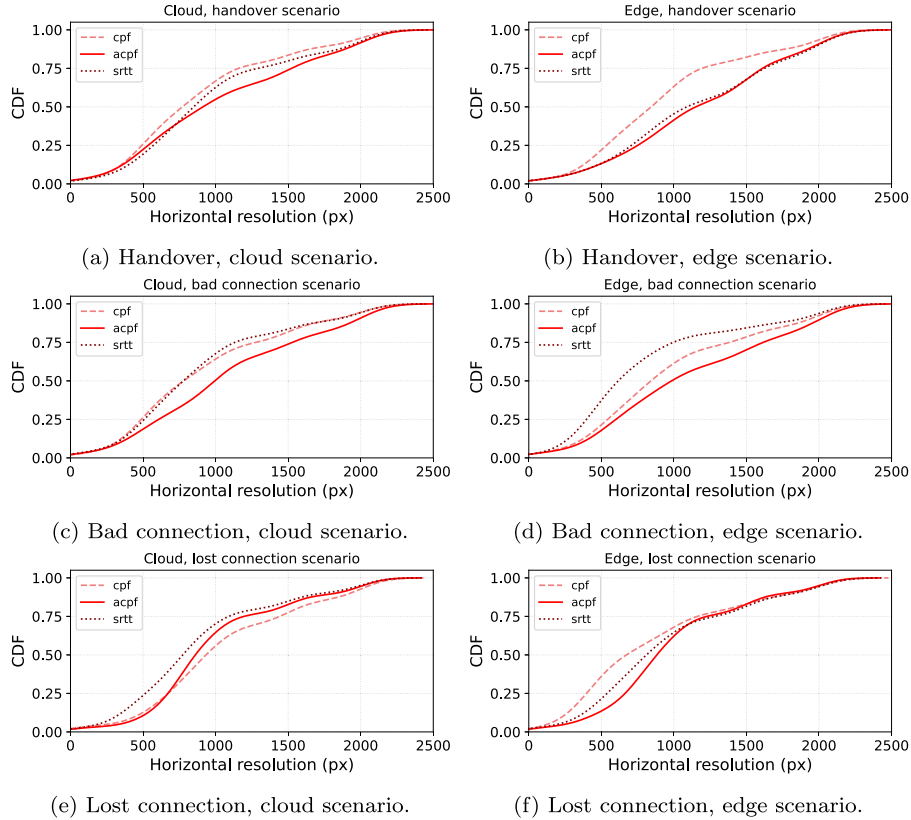
ACPF will utilize a congested path more aggressively, if there are no uncongested paths available. Fig. 10 compares two additional scenarios which are not include in Table 2: the first one has a secondary path that is sufficient to handle the 40 Mbps flow, while the second scenario has a dynamic secondary path based on LTE throughput traces which will be unable to hold the entire flow. Both scenarios are with BBR tunnels, and with the cloud deployment. During periods when aggregation is required, we expect a longer delay over both paths, as ACPF should be willing to compete for the primary path under such conditions. This is indeed what we see in Fig. 10(b), with the average queuing delay over the primary path changing from 5 ms for the first scenario to 19 ms for the second one. Although the 14 ms difference may seem small, keep in mind that these 14 ms represent larger delay spikes that only occur during approximately 50% of the flows duration, during periods when aggregation is required.

6.4. Application QoE

ACPF reduces the risk of the SRT circuit-breaker firing. Fig. 8(c) shows that *full data delivery* is almost always achieved when using ACPF or SRTT with DCCP tunnels. However, looking in Table 2 we see that a reduction in data delivery is common for the CPF/SRTT-combination, especially in the weak WiFi signal scenario. The SRT default maximum accepted transmission delay of 120 ms seems to determine the likelihood of a premature disconnect, and we see fewer



(a) Proxy in the cloud, WiFi handover trace, ACPF, static 100 Mbps secondary path.

(b) Proxy in the Cloud, WiFi handover trace, ACPF, dynamic secondary LTE path, $\sim 10 - 25$ Mbps.**Fig. 10.** ACPF mode behavior. Secondary path is: non-congested (a); congested (b).**Fig. 11.** WebRTC: CDFs of horizontal resolution, for all schedulers & scenarios.

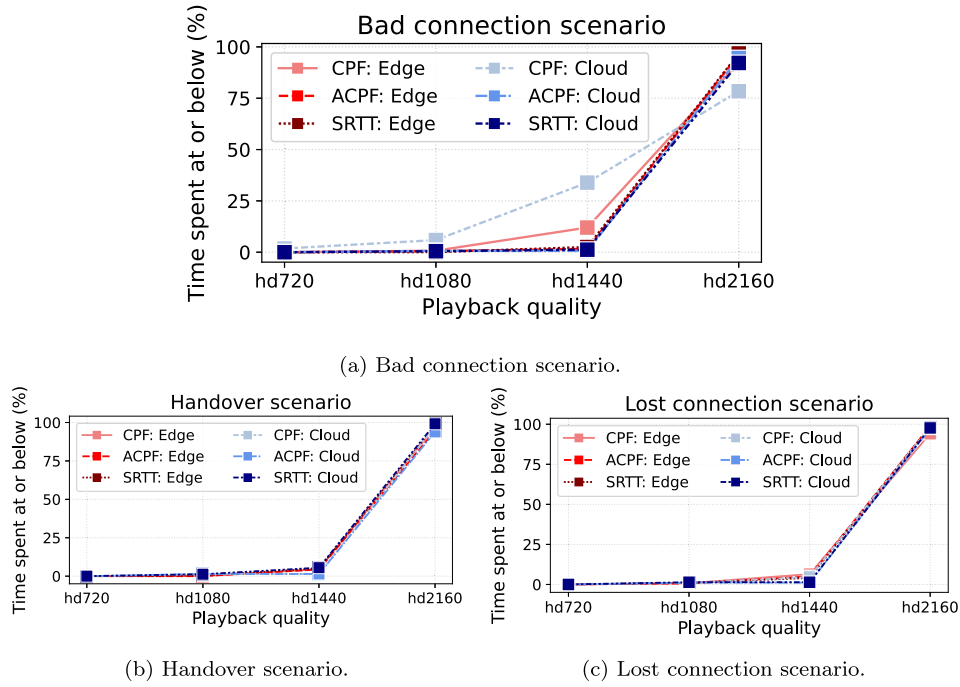


Fig. 12. YouTube: Time spent playing at or below each video quality setting, for all schedulers & scenarios.

disconnects when we e.g. increase this limit to 3000 ms. Moreover, TCP tunnels often entail a higher delay, and we consequently also see the circuit breaker firing more often and earlier when TCP tunnels are used. Finally, when adding path loss, BBR tunnels reduce the performance further for SRT applications, even when using ACPF.

ACPF enables a higher video resolution when using WebRTC. Fig. 11 shows CDF plots of the horizontal video pixel resolution for each scheduler, deployment setup and scenario. ACPF tends to provide a higher resolution than the other schedulers in a majority of the scenarios, and performs no worse than the SRTT scheduler. Only in the lost connection, cloud deployment scenario in Fig. 11(e) does CPF provide better overall performance than ACPF. Nonetheless, ACPF still performs better than SRTT in this scenario.

ACPF enables a higher resolution from YouTube in challenging scenarios. Fig. 12 shows how much time was spent playing at or below a specific video quality for each scenario and deployment over the course of 60 s. The expectation is that CPF may compete too hard for the WiFi link, when the LTE path is sufficient for the YouTube stream. As is shown in Figs. 12(b) and 12(c), the performance of all three schedulers tend to be very similar; the video spends most time playing at the maximum *hd2160* (4K) setting. Only in the *bad connection* scenario do we see a noticeable performance difference between any of the schedulers, where CPF provides a worse experience than the other two schedulers; for the cloud deployment, CPF spends roughly 22% of the time stalling — as seen by the difference between the top-right plot-point and the 100% mark.

7. Discussion & conclusion

Aggregating multiple wireless access networks using a multipath framework that schedules traffic over multiple transport layer congestion controlled tunnels is challenging. This is because of potential bad interactions of the nested congestion controls. When the path capacities fluctuate due to e.g. changes in wireless link capacities, the problem is even more difficult. This is because fast adaptation to the new characteristics is required in order to maintain low latency and high path aggregation capabilities. Much of this challenge is directed at the packet scheduler, which has to be responsive to frequent changes in channel capacity.

In this paper, we have demonstrated the need for an agile response from the packet scheduler, and presented an improved version of CPF — called *ACPFv2* — that utilizes the secondary path earlier in order to reduce the risk of high latency spikes, due to transient congestion on the primary path. We have shown that without such changes, aggregation will only occur at the cost of (i) *large delay spikes*, or (ii) *other adverse reactions* emanating from the transport or application layers, e.g. unnecessary video quality reductions. Conversely, we show that our scheduler improves the performance as measured by both the aforementioned metrics. It is clear that a trade-off exist between utilizing the primary path well, and achieving low latency, but our solution achieve both relatively well. Moreover, we have demonstrated that other schedulers, like SRTT, do not have the same problem as CPF. However, note that SRTT is *not* a substitute for CPF, ACPFv1, or ACPFv2 as SRTT does not have the desired preference for the cheap path. The comparison works in our setup only because the cheap path is also the shorter path, which will not be true in general.

Our future work aims to adapt to the path capacity changes with more accuracy. One possible idea is to use data-driven approaches to manage part of the scheduling. Another idea worth pursuing is to leverage cross-layer information and signaling between the packet scheduler and e.g. the link layer at the wireless bottleneck. A third approach is to interpret the observed delay in a more intelligent way, so as to not react to delay caused by e.g. wireless MAC-layer. As ACPFv2 targets low delay traffic, our evaluation also suggests that further improvements are possible to how ACPFv2 competes for congested paths when there is a *single shared buffer* at the bottleneck. Periodic tunnel probing may be useful in order to make better scheduling decisions when the primary path is congested, especially en lieu of having a starvation-preventing minimum CWND.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Marcus Pieska reports financial support was provided by Deutsche Telekom. Markus Amend & Eckard Bogenfeld reports a relationship with Deutsche Telekom that includes: employment. Markus Amend,

Eckard Bogenfeld, Anna Brunstrom, Andreas Kassler, Marcus Pieska has patent #SYSTEM AND METHOD FOR MULTIPATH TRANSMISSION (EP4080836A1) pending to Deutsche Telekom. Andreas Kassler is the area editor for ComNet journal.

Data availability

Data will be made available on request.

Acknowledgments

Partially funded by Region Värmland through DigitalWell Arena project (Dnr RV2018-678), by Knowledge Foundation of Sweden through DRIVE project (Dnr 20220072) and by Bavarian State Ministry of Education and Culture, Science and Art through the HighTech Agenda (HTA).

References

- [1] Ericsson, Mobility Report June 2022, 2022, Ericsson White Paper.
- [2] H. Wu, S. Ferlin, G. Caso, Ö. Alay, A. Brunstrom, A survey on multipath transport protocols towards 5G access traffic steering, switching and splitting, *IEEE Access* 9 (2021) 164417–164439.
- [3] 3GPP, System architecture for the 5G System (5GS), Technical Specification (TS) 23.501, 2021, 3rd Generation Partnership Project (3GPP), version 17.0.0.
- [4] A. Ford, C. Raiciu, M. Handley, O. Bonaventure, C. Paasch, TCP Extensions for Multipath Operation with Multiple Addresses, RFC 8684, RFC Editor, 2020.
- [5] 3GPP, Study on Access Traffic Steering, Switch and Splitting support in the 5G system architecture Phase 2, Technical Specification (TS) 23.700-93, (23.700-93) 2021, 3rd Generation Partnership Project (3GPP), version 2.0.0.
- [6] M. Amend, E. Bogenfeld, M. Cvjetkovic, V. Rakocevic, M. Pieska, A. Kassler, A. Brunstrom, A Framework for Multiaccess Support for Unreliable Internet Traffic using Multipath DCCP, in: 2019 IEEE 44th Conference on Local Computer Networks, LCN, IEEE, Osnabrueck, Germany, 2019, pp. 316–323, <http://dx.doi.org/10.1109/LCN44214.2019.8990746>.
- [7] T. Viernickel, A. Froemmgen, A. Rizk, B. Koldehofe, R. Steinmetz, Multipath QUIC: A Deployable Multipath Transport Protocol, in: 2018 IEEE International Conference on Communications, ICC, IEEE, Kansas City, MO, USA, 2018, pp. 1–7, <http://dx.doi.org/10.1109/ICC.2018.8422951>.
- [8] Y. Liu, Y. Ma, Q.D. Coninck, O. Bonaventure, C. Huitema, M. Kuhlewind, Multipath Extension for QUIC, Internet-Draft draft-ietf-quic-multipath-05, IETF Secretariat, 2023.
- [9] T. Pauly, E. Kinnear, D. Schinazi, An Unreliable Datagram Extension to QUIC, Internet-Draft draft-ietf-quic-datagram-02, Internet Engineering Task Force, 2021, Work in Progress. URL <https://datatracker.ietf.org/doc/html/draft-ietf-quic-datagram-02>.
- [10] J.R. Iyengar, P.D. Amer, R. Stewart, Concurrent multipath transfer using SCTP multihoming over independent end-to-end paths, *IEEE/ACM Trans. Netw.* 14 (5) (2006) 951–964.
- [11] O. Bonaventure, M. Piraux, Q.D. Coninck, M. Baerts, C. Paasch, M. Amend, Multipath schedulers, Internet-Draft draft-bonaventure-icrg-schedulers-00, IETF Secretariat, 2020, URL <https://www.ietf.org/archive/id/draft-bonaventure-icrg-schedulers-00.txt>.
- [12] H. Wu, Ö. Alay, A. Brunstrom, S. Ferlin, G. Caso, Peekaboo: Learning-based multipath scheduling for dynamic heterogeneous environments, *IEEE J. Sel. Areas Commun.* 38 (10) (2020) 2295–2310.
- [13] P. Hurtig, K.-J. Grinnemo, A. Brunstrom, S. Ferlin, Ö. Alay, N. Kuhn, Low-latency scheduling in MPTCP, *IEEE/ACM Trans. Netw.* 27 (1) (2019) 302–315, <http://dx.doi.org/10.1109/TNET.2018.2884791>.
- [14] M. Pieska, A. Rabitsch, A. Brunstrom, A. Kassler, M. Amend, Adaptive cheapest path first scheduling in a transport-layer multi-path tunnel context, in: Proceedings of the Applied Networking Research Workshop, Association for Computing Machinery, New York, NY, USA, 2021, pp. 39–45.
- [15] M. Zhang, M. Polese, M. Mezzavilla, J. Zhu, S. Rangan, S. Panwar, M. Zorzi, Will TCP work in mmWave 5G cellular networks? *IEEE Commun. Mag.* 57 (1) (2019) 65–71.
- [16] Z. Krämer, M. Kuhlewind, M. Ihlar, A. Mihály, Cooperative performance enhancement using QUIC tunneling in 5G cellular networks, in: Proceedings of the Applied Networking Research Workshop, Association for Computing Machinery, New York, NY, USA, 2021, pp. 49–51.
- [17] 3GPP, 5G System; Access Traffic Steering, Switching and Splitting (ATSSS); Stage 3, Technical Specification (TS) 24.193, 2021, 3rd Generation Partnership Project (3GPP), version 17.2.0.
- [18] C. Paasch, S. Ferlin, Ö. Alay, O. Bonaventure, Experimental Evaluation of Multipath TCP Schedulers, in: Proceedings of the 2014 ACM SIGCOMM Workshop on Capacity Sharing Workshop, CSWS '14, Association for Computing Machinery, New York, NY, USA, 2014, pp. 27–32, <http://dx.doi.org/10.1145/2630088.2631977>.
- [19] C. Raiciu, C. Paasch, S. Barre, A. Ford, M. Honda, F. Duchene, O. Bonaventure, M. Handley, How Hard Can It Be? Designing and Implementing a Deployable Multipath TCP, in: 9th USENIX Symposium on Networked Systems Design and Implementation, NSDI 12, USENIX Association, San Jose, CA, 2012, pp. 399–412, URL <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/raiciu>.
- [20] F. Yang, Q. Wang, P.D. Amer, Out-of-Order Transmission for In-Order Arrival Scheduling for Multipath TCP, in: 2014 28th International Conference on Advanced Information Networking and Applications Workshops, 2014, pp. 749–752, <http://dx.doi.org/10.1109/WAINA.2014.122>.
- [21] S. Ferlin, Ö. Alay, O. Mehani, R. Boreli, BLEST: Blocking estimation-based MPTCP scheduler for heterogeneous networks, in: 2016 IFIP Networking Conference (IFIP Networking) and Workshops, IEEE, Vienna, Austria, 2016, pp. 431–439.
- [22] Y.-s. Lim, E.M. Nahum, D. Towsley, R.J. Gibbens, ECF: An MPTCP Path Scheduler to Manage Heterogeneous Paths, in: Proceedings of the 13th International Conference on Emerging Networking EXperiments and Technologies, CoNEXT '17, Association for Computing Machinery, New York, NY, USA, 2017, pp. 147–159, <http://dx.doi.org/10.1145/3143361.3143376>.
- [23] Y.E. Guo, A. Nikraves, Z.M. Mao, F. Qian, S. Sen, Accelerating Multipath Transport Through Balanced Subflow Completion, in: Proceedings of the 23rd Annual International Conference on Mobile Computing and Networking, MobiCom '17, Association for Computing Machinery, New York, NY, USA, 2017, pp. 141–153, <http://dx.doi.org/10.1145/3117811.3117829>.
- [24] M. Allman, V. Paxson, W. Stevens, TCP Congestion Control, RFC 2581, RFC Editor, 1999.
- [25] S. Ha, I. Rhee, L. Xu, CUBIC: A new TCP-friendly high-speed TCP variant, *SIGOPS Oper. Syst. Rev.* 42 (5) (2008) 64–74, <http://dx.doi.org/10.1145/1400097.1400105>.
- [26] N. Cardwell, Y. Cheng, C.S. Gunn, S.H. Yeganeh, V. Jacobson, BBR: Congestion-based congestion control: Measuring bottleneck bandwidth and round-trip propagation time, *Queue* 14 (5) (2016) 20–53.
- [27] N.R. Moreno, M. Amend, A. Brunstrom, A. Kassler, V. Rakocevic, CCID5: An implementation of the BBR congestion control algorithm for DCCP and its impact over multi-path scenarios, in: Proceedings of the Applied Networking Research Workshop, 2021, pp. 52–58.
- [28] S. Ferlin-Oliveira, T. Dreibholz, Ö. Alay, Tackling the challenge of bufferbloat in Multi-Path Transport over heterogeneous wireless networks, in: 2014 IEEE 22nd International Symposium of Quality of Service, IWQoS, IEEE, Hong Kong, China, 2014, pp. 123–128, <http://dx.doi.org/10.1109/IWQoS.2014.6914310>.
- [29] C. Raiciu, M.J. Handley, D. Wischik, Coupled Congestion Control for Multipath Transport Protocols, RFC 6356, RFC Editor, 2011, <http://dx.doi.org/10.17487/RFC6356>, Request for Comments. URL <https://www.rfc-editor.org/info/rfc6356>.
- [30] K. Kaur, J. Singh, N.S. Ghuman, Mininet as software defined networking testing platform, in: International Conference on Communication, Computing & Systems, ICCS, 2014, pp. 139–142.
- [31] M. Kerrisk, Tc-netem(8) — Linux manual page, 2011, Linux man-pages project. URL <https://man7.org/linux/man-pages/man8/tc-netem.8.html>.
- [32] M. Kerrisk, Tc-tbf(8) — Linux manual page, 2001, Linux man-pages project. URL <https://man7.org/linux/man-pages/man8/tc-tbf.8.html>.
- [33] M. Sharabayko, M. Sharabayko, J. Dube, J. Kim, J. Kim, The SRT Protocol, Internet-Draft draft-sharabayko-mops-srt-01, IETF Secretariat, 2020, URL <https://www.ietf.org/archive/id/draft-sharabayko-mops-srt-01.txt>.



Marcus A. Pieska received his M.Sc. degree in computer science from Karlstad University in 2018 and is currently pursuing his Ph.D. in computer networking at Karlstad University.

His research has previously focused on transport-layer protocol performance in cellular networks, and most recently on transport-layer multipath tunneling in the same context.



Alexander Rabitsch received his M.Sc. degree in computer science from Karlstad University in 2018 and is currently working towards a PhD degree in computer science at Karlstad University.

His main research interests include transport protocols, multi-connectivity, and low-latency communication.



Anna Brunstrom received a Ph.D. and M.Sc. in Computer Science from College of William & Mary, VA, in 1996 and 1993, respectively. She received a B.Sc. in Computer Science and Mathematics from Pepperdine University, CA, in 1991.

She is currently a Full Professor in Computer Science and Head of the Distributed Intelligent Systems and Communications Research Group (DISCO) at Karlstad University, Sweden. She is also a part-time Distinguished Researcher in the Institute of Software Engineering and Technologies (ITIS) at University of Malaga, Spain. Her research interests include Internet architectures and protocols, techniques for low latency communication, multi-path communication and design and performance evaluation of mobile systems including 5G/6G.



Andreas J. Kassler received his M.Sc. degree in mathematics & computer science from Augsburg University, Germany in 1995 and his Ph.D. degree in computer science from University of Ulm, Germany in 2002.

He is currently a Full Professor in Computer Science at Deggendorf Institute of Technology at Deggendorf, Germany. He is also a Full Professor in Computer Science at Karlstad University, Sweden. His research interests include Software Defined Networks, Network Function Virtualization, Internet architecture and protocols, design and performance evaluation of mobile systems including 5G/6G and network configuration and management.



Markus Amend has a Diploma Degree in Informations and Telecommunications engineering from Technische Hochschule Mittelhessen. Since 2013, he is a researcher and strategist at Deutsche Telekom and heads the Core & Network Services unit of Group Technology since 2022.

He is active in standardization, open source development, prototyping and especially algorithms/protocols for multi-connectivity solutions. In the area of multipath network protocols he is experienced with 3GPP ATSSS, MPTCP and is the main driver of MP-DCCP.



Eckard Bogenfeld received his diploma and PhD in Electrical Engineering from the University of Kaiserslautern (Germany) in 1987 and 1992, respectively. Since 1993 he has been working in various areas of radio communications research in the Deutsche Telekom Group, including standardization and nationally and internationally funded R&D projects.

Dr. Bogenfeld currently acts in the Group Technology unit of “Technology & Innovation” with a focus on wireless networking, network optimization and multi-connectivity solutions.